

Prompt:

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements, which are the same as Assignment 2 (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create objects from scratch. Do not include pre-existing heap modules.
- Test File format:

Command Description

EMAIL <sender category>,

<subject line>,

<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

Claude (accessed by website):

// CEO Email Prioritizer - MaxHeap priority queue (array-based, built from scratch) //

Compile: g++ -std=c++17 -o ceo_email ceo_email.cpp // Run: ./ceo_email testfile.txt (or
run with no args to be prompted)

```
#include #include #include #include
```

```
// ----- Email class Email { private: std::string  
sender; std::string subject; std::string date; // MM-DD-YYYY int senderRank; // higher =  
read sooner int dateKey; // YYYYMMDD, larger = newer long sequence; // arrival order, used  
as final tie-breaker
```

```
static int rankFor(const std::string& s) {  
    if (s == "Boss")          return 5;  
    if (s == "Subordinate")    return 4;  
    if (s == "Peer")          return 3;  
    if (s == "ImportantPerson") return 2;  
    return 1;                  // OtherPerson  
}
```

```
static int keyFor(const std::string& d) {  
    // MM-DD-YYYY -> YYYYMMDD  
    int mm = std::stoi(d.substr(0, 2));  
    int dd = std::stoi(d.substr(3, 2));  
    int yy = std::stoi(d.substr(6, 4));  
    return yy * 10000 + mm * 100 + dd;  
}
```

```
public: Email() : senderRank(0), dateKey(0), sequence(0) {}
```

```
Email(const std::string& s, const std::string& subj,  
      const std::string& d, long seq)  
    : sender(s), subject(subj), date(d),  
      senderRank(rankFor(s)), dateKey(keyFor(d)), sequence(seq) {}
```

```
const std::string& getSender() const { return sender; }  
const std::string& getSubject() const { return subject; }  
const std::string& getDate()    const { return date; }
```

```

// true if this email should be read BEFORE other
bool hasHigherPriorityThan(const Email& other) const {
    if (senderRank != other.senderRank) return senderRank >
other.senderRank;
    if (dateKey != other.dateKey)      return dateKey >
other.dateKey; // newest first
    return sequence < other.sequence;  // earlier arrival wins ties
}

};

// ----- MaxHeap class MaxHeap { private:
Email* data; int capacity; int count;

static int parent(int i) { return (i - 1) / 2; }
static int left(int i)   { return 2 * i + 1; }
static int right(int i)  { return 2 * i + 2; }

void grow() {
    int newCap = capacity * 2;
    Email* bigger = new Email[newCap];
    for (int i = 0; i < count; ++i) bigger[i] = data[i];
    delete[] data;
    data = bigger;
    capacity = newCap;
}

void swapAt(int a, int b) {
    Email tmp = data[a];
    data[a] = data[b];
    data[b] = tmp;
}

void siftUp(int i) {
    while (i > 0 && data[i].hasHigherPriorityThan(data[parent(i)])) {
        swapAt(i, parent(i));
        i = parent(i);
    }
}
}

```

```

void siftDown(int i) {
    while (true) {
        int best = i;
        int l = left(i), r = right(i);
        if (l < count && data[l].hasHigherPriorityThan(data[best]))
            best = l;
        if (r < count && data[r].hasHigherPriorityThan(data[best]))
            best = r;
        if (best == i) break;
        swapAt(i, best);
        i = best;
    }
}

```

```

public: MaxHeap() : data(new Email[16]), capacity(16), count(0) {} ~MaxHeap() { delete[]
data; }

```

```

// Rule of three: prevent accidental shallow copies
MaxHeap(const MaxHeap&) = delete;
MaxHeap& operator=(const MaxHeap&) = delete;

```

```

bool isEmpty() const { return count == 0; }
int size() const { return count; }

```

```

void insert(const Email& e) {
    if (count == capacity) grow();
    data[count] = e;
    siftUp(count);
    ++count;
}

```

```

const Email& peekMax() const { return data[0]; } // call only
if !isEmpty()

```

```

void removeMax() {
    if (count == 0) return;
    data[0] = data[count - 1];
    --count;
}

```

```

        if (count > 0) siftDown(0);
    }

};

// ----- CEOInbox class CEOInbox { private:
MaxHeap heap; long nextSequence;

public: CEOInbox() : nextSequence(0) {}

void addEmail(const std::string& sender, const std::string& subject,
              const std::string& date) {
    heap.insert(Email(sender, subject, date, nextSequence++));
}

void next() const {
    if (heap.isEmpty()) {
        std::cout << "There are no emails to read." << std::endl;
        return;
    }
    const Email& e = heap.peekMax();
    std::cout << "Next email:" << std::endl;
    std::cout << "Sender: " << e.getSender() << std::endl;
    std::cout << "Subject: " << e.getSubject() << std::endl;
    std::cout << "Date: " << e.getDate() << std::endl;
}

void read() {
    heap.removeMax(); // silently does nothing if empty
}

void count() const {
    std::cout << "There are " << heap.size() << " emails to read." <<
std::endl;
}

};

```

```

// ----- Command processing class
CommandProcessor { private: CEOInbox inbox;

static std::string trim(const std::string& s) {
    size_t b = s.find_first_not_of(" \t\r\n");
    if (b == std::string::npos) return "";
    size_t e = s.find_last_not_of(" \t\r\n");
    return s.substr(b, e - b + 1);
}

void handleEmail(const std::string& args) {
    std::stringstream ss(args);
    std::string sender, subject, date;
    std::getline(ss, sender, ',');
    std::getline(ss, subject, ',');
    std::getline(ss, date);
    inbox.addEmail(trim(sender), trim(subject), trim(date));
}

public: void process(std::istream& in) { std::string line; while (std::getline(in, line)) { line =
trim(line); if (line.empty()) continue;

        if (line.rfind("EMAIL ", 0) == 0) {
            handleEmail(line.substr(6));
        } else if (line == "NEXT") {
            inbox.next();
        } else if (line == "READ") {
            inbox.read();
        } else if (line == "COUNT") {
            inbox.count();
        }
    }
}

};

```

```
// ----- main
int main(int argc, char* argv[])
{
    std::string filename;
    if (argc > 1) { filename = argv[1]; }
    else { std::cout << "Enter test file name: ";
           std::getline(std::cin, filename); }

    std::ifstream file(filename);
    if (!file) {
        std::cerr << "Could not open file: " << filename << std::endl;
        return 1;
    }

    CommandProcessor processor;
    processor.process(file);
    return 0;
}
```

Gemini (accessed by website):

```
#include <iostream>
```

```
#include <string>
```

```
// Value object for Date parsing and comparison
```

```
class EmailDate {
```

```
private:
```

```
    int month, day, year;
```

```
public:
```

```
    EmailDate() : month(0), day(0), year(0) {}
```

```
    EmailDate(const std::string& dateStr) {
```

```
        if (dateStr.length() == 10) {
```

```
            month = std::stoi(dateStr.substr(0, 2));
```

```
            day = std::stoi(dateStr.substr(3, 2));
```

```
            year = std::stoi(dateStr.substr(6, 4));
```

```
        } else {
```

```
            month = day = year = 0;
```

```
        }
```

```
    }
```

```
    bool isNewerThan(const EmailDate& other) const {
```

```
        if (year != other.year) return year > other.year;
```

```
        if (month != other.month) return month > other.month;
```

```
        return day > other.day;
```

```
    }
```

```
};
```

```
// Domain Object representing Priority Rules
```

```
class PriorityEvaluator{
```

```
public:
```

```
    static int getCategoryRank(const std::string& category) {
```

```
        if (category == "Boss") return 5;
```

```
        if (category == "Subordinate") return 4;
```

```
        if (category == "Peer") return 3;
```

```
        if (category == "ImportantPerson") return 2;
```

```
        if (category == "OtherPerson") return 1;
```

```
        return 0;
```

```
    }
```

```
};
```

```
// CEO Email Entity
```

```
class CEOEmail {
```

```
private:
```

```
    std::string senderCategory;
```

```
    std::string subjectLine;
```

```
    std::string dateString;
```

```
    EmailDate parsedDate;
```

```
public:
```

```
    CEOEmail() {}
```

```
    CEOEmail(std::string category, std::string subject, std::string date)
```

```
    : senderCategory(category), subjectLine(subject), dateString(date), parsedDate(date) {}
```

```
std::string getSenderCategory() const { return senderCategory; }
```

```
std::string getSubjectLine() const { return subjectLine; }
```

```
std::string getDateString() const { return dateString; }
```

```
bool hasHigherPriorityThan(const CEOEmail& other) const {
```

```
    int rankThis = PriorityEvaluator::getCategoryRank(senderCategory);
```

```
    int rankOther = PriorityEvaluator::getCategoryRank(other.senderCategory);
```

```
    if (rankThis != rankOther) {
```

```
        return rankThis > rankOther;
```

```
    }
```

```
    // Sprint rule: Newer emails take priority over older emails
```

```
    return parsedDate.isNewerThan(other.parsedDate);
```

```
}
```

```
};
```

```
// Doubly Linked List Node for Max Heap
```

```
class PriorityNode {
```

```
public:
```

```
    CEOEmail payload;
```

```
    PriorityNode* nextNode;
```

```
    PriorityNode* prevNode;
```

```
    PriorityNode(CEOEmail email)
```

```
        : payload(email), nextNode(nullptr), prevNode(nullptr) {}  
};
```

```
// List-Based MaxHeap Queue
```

```
class PriorityQueueHeap {
```

```
private:
```

```
    PriorityNode* headNode;
```

```
    int count;
```

```
    PriorityNode* fetchNodePointerAt(int index) {
```

```
        PriorityNode* current = headNode;
```

```
        int currentIdx = 0;
```

```
        while (current != nullptr && currentIdx < index) {
```

```
            current = current->nextNode;
```

```
            currentIdx++;
```

```
        }
```

```
        return current;
```

```
    }
```

```
    void shiftUp(int index) {
```

```
        while (index > 0) {
```

```
            int parentIdx = (index - 1) / 2;
```

```
            PriorityNode* child = fetchNodePointerAt(index);
```

```
            PriorityNode* parent = fetchNodePointerAt(parentIdx);
```

```
            if (child->payload.hasHigherPriorityThan(parent->payload)) {
```

```

        CEOEmail temp = parent->payload;
        parent->payload = child->payload;
        child->payload = temp;
        index = parentIdx;
    } else {
        break;
    }
}
}

```

```

void shiftDown(int index) {
    while (index < count) {
        int leftChildIdx = 2 * index + 1;
        int rightChildIdx = 2 * index + 2;
        int priorityIndex = index;

        PriorityNode* highestPriorityNode = fetchNodePointerAt(priorityIndex);

        if (leftChildIdx < count) {
            PriorityNode* leftNode = fetchNodePointerAt(leftChildIdx);
            if (leftNode->payload.hasHigherPriorityThan(highestPriorityNode->payload)) {
                priorityIndex = leftChildIdx;
                highestPriorityNode = leftNode;
            }
        }
    }
}

```

```

    if (rightChildIdx < count) {
        PriorityNode* rightNode = fetchNodePointerAt(rightChildIdx);
        if (rightNode->payload.hasHigherPriorityThan(highestPriorityNode->payload)) {
            priorityIndex = rightChildIdx;
            highestPriorityNode = rightNode;
        }
    }

    if (priorityIndex != index) {
        PriorityNode* currentNode = fetchNodePointerAt(index);
        CEOEmail temp = currentNode->payload;
        currentNode->payload = highestPriorityNode->payload;
        highestPriorityNode->payload = temp;

        index = priorityIndex;
    } else {
        break;
    }
}
}

```

public:

```
PriorityQueueHeap() : headNode(nullptr), count(0) {}
```

```
~PriorityQueueHeap() {
```

```
    PriorityNode* current = headNode;
```

```

while (current != nullptr) {
    PriorityNode* next = current->nextNode;
    delete current;
    current = next;
}
}

```

```

void push(const CEOEmail& email) {
    PriorityNode* newNode = new PriorityNode(email);
    if (headNode == nullptr) {
        headNode = newNode;
    } else {
        PriorityNode* lastNode = fetchNodePointerAt(count - 1);
        lastNode->nextNode = newNode;
        newNode->prevNode = lastNode;
    }
    count++;
    shiftUp(count - 1);
}

```

```

CEOEmail peek() {
    if (count > 0) {
        return headNode->payload;
    }
    return CEOEmail();
}

```

```

void pop() {
    if (count == 0) return;

    PriorityNode* lastNode = fetchNodePointerAt(count - 1);
    headNode->payload = lastNode->payload;

    if (lastNode->prevNode != nullptr) {
        lastNode->prevNode->nextNode = nullptr;
    } else {
        headNode = nullptr;
    }

    delete lastNode;
    count--;

    if (count > 0) {
        shiftDown(0);
    }
}

int getCount() const { return count; }
bool isEmpty() const { return count == 0; }
};

```

// Command Controller Object

```

class CEOInboxController {
private:
    PriorityQueueHeap queue;

public:
    void execute(const std::string& commandLine) {
        if (commandLine.empty()) return;

        if (commandLine.rfind("EMAIL", 0) == 0) {
            size_t firstSpace = commandLine.find(' ');
            std::string content = commandLine.substr(firstSpace + 1);

            size_t c1 = content.find(',');
            size_t c2 = content.find(',', c1 + 1);

            std::string sender = content.substr(0, c1);
            std::string subject = content.substr(c1 + 1, c2 - c1 - 1);
            std::string date = content.substr(c2 + 1);

            CEOEmail newEmail(sender, subject, date);
            queue.push(newEmail);
        }
        else if (commandLine == "COUNT") {
            std::cout << "There are " << queue.getCount() << " emails to read.\n";
        }
        else if (commandLine == "NEXT") {

```

```

        if (!queue.isEmpty()) {
            CEOEmail email = queue.peek();
            std::cout << "Next email:\n";
            std::cout << "Sender: " << email.getSenderCategory() << "\n";
            std::cout << "Subject: " << email.getSubjectLine() << "\n";
            std::cout << "Date: " << email.getDateString() << "\n";
        }
    }
    else if (commandLine == "READ") {
        if (!queue.isEmpty()) {
            queue.pop();
        }
    }
}

};

int main() {
    CEOInboxController controller;

    std::string testFileCommands[] = {
        "EMAIL Peer,Can you help me on this?,12-01-2024",
        "EMAIL OtherPerson,Try our product,12-19-2024",
        "EMAIL Boss,Important,12-20-2024",
        "EMAIL Subordinate,How do I handle this?,12-25-2024",
        "EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024",
        "EMAIL Boss,Never Mind,01-03-2025",
    }
}

```

"COUNT",

"NEXT",

"READ",

"NEXT",

"READ",

"COUNT"

};

for (const std::string& line : testFileCommands) {

 controller.execute(line);

}

return 0;

}

Analysis:

Correctness:

Claude:

Doesn't compile properly, unsure about edge cases or error handling.

Gemini:

Compiles, runs, but doesn't accept file input. Has a preprogrammed input that runs correctly. I am unsure about edge cases or error handling.

Time complexity:

Claude:

No nested for loops or recursive calls. The program is unable to run for benchmarking.

Gemini:

No nested loops or recursive calls. Runs in 0.004s given any input due to the file input not working.

Space Complexity:

Comparison:

Both use the same amount of while loops, as well as a for loop to read the file. There are no recursive functions in either program. I am unable to benchmark the code from Claude, but Gemini used a peak memory of 3816kb.

Maintainability:

Claude:

Limited comments, only explaining what the classes are and some of the values in them. Variable names are logical, and there are no repeated blocks of code.

Gemini:

Even less comments, only separating sections of code. Variable names are logical, and there are no repeated blocks of code.

Selection:

Gemini is a clear winner as the code from Claude fails to compile.

Changes:

Correctness:

I will add the ability to run with file input and error handling for incorrect file inputs.

Maintainability:

I will add both a prologue comment and comments on every line of code explaining what they do.